

Reinforcement Learning in Bipedal Robots

Mohamed Ali Dammak

June 25, 2025

A technical report



Contents

1	Introduction	2
2	Basics of Reinforcement Learning	2
2.1	Markov Decision Processes	2
2.2	Agent-environment interaction loop	2
2.3	Common algorithms	2
2.3.1	PPO	3
2.3.2	DDPG	3
2.3.3	SAC	3
2.4	Exploration and Exploitation	4
2.4.1	Exploitation	4
2.4.2	Exploration	4
2.4.3	Balancing Exploration and Exploitation	4
3	RL Environments	4
3.1	Agent	4
3.2	Action and States	5
3.3	Environment	5
3.4	Reward Function Design	5
4	Key Physical Characteristics In Bipedal Robots	6
4.1	Center of Mass	6
4.2	ZMP and Support Polygons	6
4.3	Capture Points	6
5	UNITREE H1 Robot	6
5.1	Hardware specifications	6
5.2	Software-Side Overview	6
6	Implementing Simulation	7
6.1	Simulation Tools	7
6.2	Preparing RL simulation	7
6.3	Interfacing the RL model with Simulator(Mujoco)	7

1 Introduction

Bipedal locomotion represents one of the most complex challenges in robotics, requiring dynamic balance, adaptive gait control, and robust environmental interaction. Traditional control methods such as Zero-moment-point are often used to control this kind of locomotion.

Reinforcement learning is a trial and error learning paradigm that a machine can use by maximizing a specific signal. This report examines the application of RL to bipedal robots, focusing on environment setup, reward design considerations, and simulation strategies. Key challenges include maintaining stability during gait transitions, handling underactuated dynamics.

2 Basics of Reinforcement Learning

Reinforcement learning is a learning paradigm in which an Agent interacts with an environment, and based on the state or action, the Agent receives a "Reward"; The "Reward" can be a positive value or a negative value, the Goal of the agent to maximize this Reward. The long-term actions of the agent are determined by a "Reward function", this is used to "guide" the agent towards a desired goal.

2.1 Markov Decision Processes

The Markov Decision is a mathematical model the reinforcement learning is built upon. In where there exists State Space and there exists and action space. The state is the Set all different values of a vector of Sensory inputs (in the context of a robot), While the action space is the different sets of actions that can be taken. There is a probability calculation, that determines the probability of moving from one state to another under a specific action and the corresponding reward. From these, The agent learns a new policy (how this policy changes is determined by the selected algorithm) The goal of a reinforcement learning agent is to learn a policy that maximizes the expected cumulative reward.

2.2 Agent-environment interaction loop

A key component in the RL paradigm is the concept an episode; an episode can be defined through a time limit, or when a condition is true. In each of these episodes there are several iterations of the loop between the agent and environment:

- The Agent starts in a state, this state is then observed, this is the **Initial State and Observation**
- Agent decides action based on policy parameters
- Environment Executes Action
- Environment Computes **reward**

2.3 Common algorithms

Three of the most common algorithms in reinforced learning are PPO DDPG and SAC

2.3.1 PPO

PPO learns from data generated by the current policy being evaluated. This means the agent explores the environment and collects experience, and then uses that experience to improve its policy. PPO typically uses an actor-critic architecture, where the "actor" selects actions based on the current policy, and the critic evaluates the chosen actions, providing feedback to improve the

Pros:

- Stable and reliable: Minimizes drastic policy updates (uses clipping).
- Works for discrete/continuous actions.
- Low variance: Good empirical results across environments.

Cons:

- On-policy: Cannot reuse old data (less sample-efficient).
- Computationally heavy: Requires more environment interactions.
- Sensitive to clipping range: Poor tuning hurts performance.

2.3.2 DDPG

Deep Deterministic Policy Gradient (DDPG) is a reinforcement learning algorithm designed for environments with continuous action spaces. It combines the strengths of both policy-based and value-based methods by employing an actor-critic architecture. The "actor" network learns a deterministic policy that maps states to specific actions, while the "critic" network evaluates the chosen actions by estimating their Q-values

Pros:

- Handles continuous actions (e.g., robotics, driving).
- Off-policy: Efficient data reuse (replays experiences).
- High sample efficiency (learns quickly with fewer samples).

Cons:

- Hyperparameter sensitivity: Tuning is tricky (e.g., noise, learning rates).
- Unstable training: Prone to divergence if not carefully managed.
- Poor exploration: Relies on manual noise tuning (e.g., OU noise).

2.3.3 SAC

It's an off-policy algorithm that combines actor-critic methods with entropy regularization to learn a stochastic policy that maximizes both expected reward and policy entropy. This means the agent learns to perform well on the task while also acting as randomly as possible, promoting exploration and preventing premature convergence to suboptimal solutions.

Pros:

- Best sample efficiency: Off-policy + entropy regularization.
- Automated exploration: Adjusts temperature to balance exploration/exploitation.
- Stable: Handles complex tasks well

Cons:

- Complexity: Harder to implement (3 networks + entropy tuning).
- Computationally expensive: More parameters per update.
- Over-exploration risk: Entropy focus may delay convergence.

2.4 Exploration and Exploitation

Exploration and exploitation are used to build learning algorithms, one working off of past information, and the testing random action and observing the results.

2.4.1 Exploitation

Exploitation is the strategy of using already known knowledge to make decisions on future actions, The key aspects of exploitation are to maximize the short term reward based on what's already known, decision efficiency, and risk aversion.

2.4.2 Exploration

Exploration is used to increase knowledge about an environment or model, it selects action with unknown states or outcomes, to gain information and minimize uncertainty, exploration makes sure that enough different areas of the state space are visited to prevent learning that is biased toward a few experiences

2.4.3 Balancing Exploration and Exploitation

It is necessary that there is balance between both exploration and exploitation actions for an efficient machine learning process, in order that rewards for actions keeps getting bigger. There are two ways this can be implemented.

The first one being the Epsilon-Greedy Strategy, by which the agent rolls the dice and has a probability to take an exploration or exploitation action and after each action the reward value is estimated as such: $Q(a) \leftarrow Q(a) + \frac{1}{N(a)} (R - Q(a))$. Another way is the use of the upper confidence Bound, which the average reward of all the actions from past attempts and adding a term for uncertainty (gets smaller every time an action is tried), the less an action is taken the more likely it is to be tested. This way in on term, both how rewarding an action was and untested it is gone on to determine its probability.

3 RL Environments

3.1 Agent

The agent in an RL environment is the decision maker in deciding what action to take, an example in the environment of someone trying to ride a bike, would be the rider,

while the bike and the landscape is the rest of the environment. The agent is made up of two main components, the policy and the Reinforcement learning Algorithm. The agent receives two observations, one of the current state and one of the next state. These states are then processed using the policy function and decides on the action. Another input the agent receives is the output of the reward function.

Within each agent, there is a learning algorithm, this algorithm receives the state, the action taken and the reward, then uses this information to update the policy.

3.2 Action and States

The values that the Agent-Environment loops rely on are the actions and the states.

States are the current configuration of the environment(e.g. position, sensor readings), at every time step, a state is read from the environment. Some states may be terminal (end the episode), not terminal.

The agent has control over many actions from a set of actions in the action space(e.g. Move left, up down right), these actions are decided by agents' policy, these action then influence the environment and affect the reward received.

3.3 Environment

The environment is what provides the Agent component with the current state and what computes the Reward function. The environment is affected by the actions the user has taken, and these actions change the state. The reward function in the environment is computed from both the state and from the action put up against the environment

3.4 Reward Function Design

Reward function is a function provides feedback on the actions done. This reward value is used to update the policy function in on-policy methods such as (PPO), and update replay buffer in off-policy algorithms. The reward function can be manipulated in order for it. The goal of the Agent is to have maximum cumulative rewards for every episode.

When designing a reward function, the user must determine the hierarchy of task; for a simpler system, the system may have one task. To order a hierarchy, tasks that have a higher priority, must have greater impact on the reward value than those values below them (whether that be positive or negative values).

There are also different ways to design a reward variable, one might have it be cumulative value (integral or sum) or have it be an

There are several things to consider for the Reward function of a bipedal robot; for this example the goal of the Robot is to walk smoothly along a path. With the aforementioned goal, the hierarchy of tasks is set: Firstly to maintain balance this can be ensured that the Center of mass does not translate along the Z-axis (Penalty for this shift), Secondly is walking forward (Reward * Forward Velocity), Thirdly is Minimizing actuator effort, this can be done by adding a penalty for the sum of all previous steps. In addition to this, the biped must walk in a straight line and be rewarded for every action taken. Therefore, the reward function will be as such: v_x : forward velocity, y : Deviation from path, $\frac{T_s}{T_F}$: Current sample time over final time of simulation, u_{t-1} : is torque of the joint from all previous steps.

$$r_t = v_x - 3y^2 - 50\hat{z}^2 + 25\frac{T_s}{T_F} - 0.02 \sum_i (u_{t-1}^i)^2$$

4 Key Physical Characteristics In Bipedal Robots

Bipedal Robots have unique physical constraints, that must take into account when designing their control: For the purposes of a smooth walking Robot, those are Centers of Mass, Support Polygons, and Capture points.

4.1 Center of Mass

The center of mass of any Object is the point of equilibrium of any object. The first and primary problem in making a walking robot, is stabilizing it, to do that one must Control the Center of Mass. to find the center of mass, 2 force/torque sensors are used at the feet of robot

4.2 ZMP and Support Polygons

Zero Moment Point (**ZMP**) is a principle used to establish robots, it is a point where the robot is making contact with the ground, where sum of all moments of the robot is equal to zero. If this point is inside the stability area, the robot will not fall down. The stability area is bounded by the point where the feet touch the ground and coincides with that support polygon.

4.3 Capture Points

A capture point is a position on the ground by which the robot can bring itself to a complete stop, without falling, or having to take another step.

5 UNITREE H1 Robot

The *UNITREE_{TM}* H1 is a general purpose humanoid (Bipedal) robot released in late 2023. The purpose of H1 Humanoid Robot is mainly for research and development meant to bridge the gap between Simulation and reality. The H1 is equipped with Depth cameras, IMU's, Lidar, Torque/Force Sensors, and Joint Position sensors.

5.1 Hardware specifications

The H1 humanoid robot presents a significant high-dimensional control challenge for Reinforcement Learning (RL) with its 39 actuated joints, including articulated hands and a waist, enabling complex whole-body movement. Powered by high-torque-density electric actuators, it achieves dynamic maneuvers like running (5 m/s), jumping, and rapid directional changes, pushing RL towards high-speed stabilization and agile recovery. A comprehensive sensor suite provides rich state information (joint encoders, IMU, foot sensors, plus optional depth/LiDAR for perception), while its physical robustness withstands falls, making it ideal for real-world RL training, deployment, and sim-to-real transfer experiments.

5.2 Software-Side Overview

The H1's accessible software architecture is critical for RL, offering unprecedented low-level control. Unlike platforms restricted by proprietary controllers, its SDK allows re-

searchers to send direct torque, position, or velocity commands to individual joints, enabling RL agents to learn fundamental locomotion skills without stabilization bypasses. Native ROS 2 integration provides essential standardized middleware for communication, data logging, sensor fusion, and real-time policy deployment, aided by a URDF model for simulation. A comprehensive SDK grants APIs for robot control, sensor data access, and system management, facilitating seamless integration with custom RL frameworks (e.g., PyTorch, TensorFlow). Unitree further supports the community by actively releasing open-source components like ROS drivers and simulation models, accelerating RL development and collaboration.

6 Implementing Simulation

6.1 Simulation Tools

MuJoCo (Multi-Joint dynamics with Contact) is a high-fidelity physics engine optimized for robotics and biomechanics. It provides accurate simulations of complex contacts, joint dynamics, and actuator models, making it ideal for simulating bipedal locomotion. Its native integration with OpenAI Gym enables standardized RL experimentation.

Isaac Lab (formerly Isaac Gym) is a GPU-accelerated robotics simulator from NVIDIA. Built on the Omni verse platform, it performs thousands of parallel simulations in real-time by leveraging GPU parallelism. This is particularly advantageous for RL training, where massive sample throughput accelerates learning for high-DoF bipedal robots.

OpenAI Gym (and its successor Gymnasium) is a standardized API for RL environments. It provides pre-built environments for bipedal robots (e.g., Humanoid-v4 in MuJoCo) with unified interfaces for actions, observations, and rewards. This allows seamless integration with RL algorithms and libraries like Stable Baselines.

6.2 Preparing RL simulation

The workflow begins with selecting a suitable bipedal environment (e.g., Humanoid-v4) that matches the robot’s dynamics, configuring observations (joint positions, velocities, torso orientation) and actions (motor torques/positions), and defining a reward function incorporating forward velocity, stability penalties, energy costs, and fall termination. Next, an algorithm is chosen based on system requirements: PPO for stable sample efficiency. Training iterations run with periodic evaluation checkpoints while monitoring reward, survival time, and velocity targets, leveraging tools like. Finally, policies are tested in varied terrains/disturbances, failure modes (foot slipping, balance loss) are analyzed, and the validated model is exported for sim-to-real deployment.

6.3 Interfacing the RL model with Simulator(Mujoco)

To interface MuJoCo for bipedal robot reinforcement learning, developers can leverage pre-configured environments, This can be done through Gymnasium’s standardized API, initialized with parameters (Through a python script). For custom robot designs, developers create XML models defining kinematic chains, joints, and actuators and defining observation/action spaces using MuJoCo’s native data structures (e.g. Joint positions). and utilizing MuJoCo’s visualization tools for real-time monitoring.